

DriftConditions Implementation Plan

Spring 2026 Operations Upgrade

1. Purpose

This implementation plan outlines the next wave of **MixEngine** improvements, backend infrastructure work, and stream quality fixes for DriftConditions. It is a companion to the Spring 2026 UI/UX Upgrade Implementation Plan and addresses work deferred from that effort — primarily items requiring **MixEngine** instrumentation, new database tables, or changes to how the station generates and manages its stream.

2. Goals and Scope

2.1. Goals

The primary goals of this upgrade are to:

1. **Improve Stream Quality and Musical Coherence:** Fix known bugs in clip selection, volume balance, duration calculation, and recipe behavior. Extend the filter system with missing capabilities.
2. **Instrument the **MixEngine** for Usage Tracking:** Record which clips are selected and played, enabling usage-weighted selection and surfacing play counts in the admin UI.
3. **Strengthen Backend Infrastructure:** Add audit logging for auth and user events, implement contributor notifications, and consolidate file-based logging.
4. **Explore Stream Expansion:** Investigate YouTube and Twitch output as a stretch goal.

2.2. Why these goals matter:

The **MixEngine** is the core of what DriftConditions does — everything the listener hears flows through it. Several known bugs degrade the quality of that experience: clips outside the requested length range are occasionally selected, volume balance between tracks is sometimes wrong, and certain recipes misbehave. Fixing these improves the station's output directly. Usage tracking closes a long-standing gap — the system currently has no memory of which clips it has played, making weighted selection and contributor feedback impossible. Infrastructure improvements reduce operational risk and lay groundwork for growth.

2.3. Scope

This upgrade focuses on server-side and **MixEngine** changes:

2.3.1. MixEngine Fixes

- Fix clip length selection — clips outside the requested range being chosen by **ClipSelector**
- Resolve intermittent NaN duration calculation
- Fix volume balance — background ambient tracks overpowering foreground elements
- Debug Long Narrative With Music Bed recipe misbehavior
- Fix normalization level on quiet clips

2.3.2. MixEngine Additions

- New filters: **fadeout**, **loop(n)**, **repeat(n)**, music bed ducking
- Recipe length weighting — prevent long environmental clips from dominating the stream
- Clip usage tracking — new **clipUsage** table; **MixEngine** instrumented to log each clip selected
- Usage-weighted clip selection — prefer less-used clips over time

2.3.3. Backend Infrastructure

- Audit table instrumentation — write auth and user change events (currently empty)
- Contributor digest email system — periodic summary of approvals and clip usage per user
- Logging consolidation — reduce or eliminate file-based logs in **log/***

2.3.4. Stretch Goals

- YouTube / Twitch stream output

2.4. Out of Scope:

- Admin UI changes beyond what is directly enabled by new backend data (clip usage display on Audio View/Edit is specified in the UI/UX plan)

- Auth/authorization system changes (covered in the Auth Upgrade plan)
- New recipe authoring functionality

3. Implementation: MixEngine Fixes

The following tasks address known defects in the core clip selection, duration, and mixing logic. Each fix targets a specific failure mode that degrades the quality of the generated stream.

3.1. Clip Length Selection Correction

The `ClipSelector` component occasionally selects clips that fall outside the length range specified by the recipe — for example, choosing a 100-second clip when the recipe requests something between 2 and 10 minutes. The selection logic applies a range filter, but the filter is not always enforced strictly, likely due to a boundary condition or fallback path that bypasses the check.

The fix requires identifying the code path where out-of-range clips are returned and adding or correcting the length constraint. The expected result is that `ClipSelector` never returns a clip whose duration falls outside the recipe's specified `minLength/maxLength` bounds, with no silent fallback exceptions.

3.2. Resolve Intermittent NaN Duration Calculation

The `MixEngine` intermittently calculates an invalid (NaN) value for track duration, producing an `ffmpeg` error logged as `Unable to parse option value "NaN"`. This causes the affected mix to fail silently or produce a malformed output.

The root cause is likely in the expression evaluator or duration math when one or more input values is undefined or zero. The fix involves adding defensive validation at the point where durations are computed — ensuring that any `NaN`, `undefined`, or non-numeric result is caught before it propagates to the `ffmpeg` filter chain.

3.3. Volume Balance Correction

In some mixes, background ambient tracks overpower foreground elements — voices, narratives, or featured sounds — producing a sonically incoherent output. The current gain assignments in the filter chain do not reliably establish a hierarchy between background and foreground layers.

This task involves reviewing the relative gain values applied to background vs. foreground tracks and adjusting them to enforce a consistent sonic hierarchy. The fix should be applied at

the recipe or filter layer so that the relationship holds across different clip combinations without per-clip tuning.

Resolution: Close as won't fix at the code level. Recipe authors should ensure ambient/background tracks are set lower than foreground tracks. If automatic balancing becomes desirable in the future, it would require adding a `role` field to the track schema and corresponding logic in MixEngine to enforce relative volume constraints.

3.4. Debug Long Narrative With Music Bed Recipe

The "Long Narrative With Music Bed" recipe is not executing its intended logic correctly. The exact failure mode is not yet characterized — the recipe may be producing incorrect structure, wrong durations, or missing elements.

This task begins with diagnosis: running the recipe in isolation and comparing its actual output structure against its intended design. Once the root cause is identified, the fix will be scoped and applied. No implementation approach is specified here until diagnosis is complete.

Root cause identified and resolved. Historical `mixQueue` records showed two mixes with anomalous 6654s (110 min) durations. Both selected the same clip — "H1Z1 Survivor Radio Broadcast" at 6413s (106 min) — which exceeded the previous `huge` maximum of 3600s (60 min). The clip was reachable because the recipe at the time contained a malformed `length` key ("`long, huge`" as a single comma-separated string rather than [`"long"`, `"huge"`]), which caused the length filter to silently fail, allowing any-duration clip to be selected.

Resolution: Recipe corrected (separate strings). `huge` category max extended to 7200s (120 min) to accommodate legitimately long-form content. Warning logging added to `_constructLengthQuery` to surface future malformed keys.

3.5. Fix Normalization Level on Quiet Clips

Some audio clips — particularly quiet field recordings and spoken-word material — are normalized to a level too low for the mix, resulting in an inaudible or barely audible contribution to the stream. The normalization filter is applying a target that is insufficient for these clips.

The fix involves adjusting the normalization target to a louder standard and verifying that the result is audible in context without clipping or distortion. This may require a per-clip override path or a louder default target level.

Resolution: Investigation revealed that the normalization targets themselves were not the root cause. The `loudnorm` presets (`voice`: `L=-16 LUFS`, `bed`: `L=-20 LUFS`) were appropriate, but `ffmpeg`'s `amix` filter was silently undermining them. `amix` defaults to `normalize=1`, which scales each input by $1/N$ — with two tracks, every track's level is halved (~ -6 dB) before the final

mix, directly undoing the per-clip normalization work. The fix detects whether any track or clip in the recipe uses a `norm` effect; if so, `amix` is set to `normalize=0`, allowing the pre-normalized levels to pass through unchanged. Recipes without norm effects retain the default behavior as a clipping safeguard. No changes to the `loudnorm` presets were required.

3.6. Fix and Improve Flexible Clip Duration Assignment

The `ClipAdjustor` uses flexible clips — those with a `minLength/maxLength` range rather than a fixed duration — as spacers between effect clips on a track. The recipe creator's choice of range expresses distribution intent:

- `track2: tiny silence + effect + tiny silence + effect + tiny silence`
→ effect clips bunched near the start; small gaps between them
- `track2: short-long silence + effect + short-long silence + effect + short-long silence`
→ effect clips spread loosely across track1's full duration

The original implementation used a random search loop targeting 33% fill of `mixDuration`. When flexible clips couldn't span that threshold (e.g. short silences alongside a 54-minute ambient track), the loop never exited, spinning until Node.js exhausted its heap and crashed MixEngine.

Fix: Replaced with rejection sampling. Each flexible clip is sampled independently and uniformly within its own `[minLength, maxLength]` range. An attempt is accepted if the total fits within the available budget. This correctly honors recipe intent — a `tiny` silence stays tiny regardless of its neighbors; a `short-long` silence can expand freely.

Edge cases:

- **Pre-check:** if even minimum durations exceed the budget (fixed content too long), all flexible clips are scaled down proportionally and the loop is skipped entirely.
- **Attempt limit:** `config.audio.silenceAdjustMaxAttempts` (default: 100) caps the loop. If exceeded, the last attempt is scaled proportionally to fit the budget — better than resetting to minimums because relative spacing between effect clips is preserved.
- **Generalized:** renamed from `_adjustSilences` to `_adjustFlexibleClips`; any clip type with `minLength/maxLength` (e.g. a future `stretch` effect) is handled automatically.

4. Implementation: MixEngine Additions

This section covers new capabilities being added to the `MixEngine` to increase stream variety, improve recipe expressiveness, and introduce the data infrastructure for usage tracking and weighted selection.

4.1. Audio Clip Usage Tracking

Recipe clips are selected dynamically at runtime by the `MixEngine` based on classification and length criteria. Individual audio clips are not referenced by ID in recipe data, so accurate usage tracking requires the `MixEngine` to instrument and log each clip it selects.

4.1.1. Infrastructure Changes

SQL

```
-- New table: tracks each clip actually heard in a mix
CREATE TABLE clipUsage (
  usageID INT NOT NULL AUTO_INCREMENT,
  audioID INT NOT NULL,
  recipeID INT NOT NULL,
  usedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (usageID),
  KEY idx_audioID (audioID),
  KEY idx_recipeID (recipeID)
);

-- New column: cached play count on audio table for fast lookups
ALTER TABLE audio ADD COLUMN timesUsed INT DEFAULT 0;
```

A new `RecordKeeper` service is inserted into the `Conductor` pipeline immediately after `ClipAdjustor.adjustClipTimings()`, at which point all clip durations and `mixDuration` are fully resolved. It walks each track accumulating elapsed time, identifies which clips fall within `mixDuration` (i.e., were actually heard), updates `audio.lastUsed` and increments `audio.timesUsed` for each heard clip, inserts a `clipUsage` row per heard clip, and returns an accurate playlist. Silence clips are excluded from logging; tiny clips are logged but excluded from the public-facing playlist.

This replaces `RecipeParser.getPlaylistFromRecipe()` and `ClipSelector._updateClipLastUsed()`, both of which operated on the full selected-clip list without accounting for mix truncation.

4.1.2. Considerations

- *clipUsage growth*: Flagged as a concern before launch. Resolved — `audio.timesUsed` serves as a counter-cache column, so `/api/audio/info` reads a single integer from `audio.*` rather than issuing a COUNT join against `clipUsage`. The detail table is retained for digest emails and analytics (3.4, 4.2).
- *Tracking at selection time vs. confirmed play time*: Selection is simpler to instrument; confirmed play requires stream feedback from Icecast. We implemented a third option — **heard time** — computed from `mixDuration` after timing adjustment. This is more accurate than selection time (clips cut off by a shorter mix are excluded) while avoiding the complexity of Icecast feedback.
- `clipUsage` also feeds usage-weighted clip selection (3.4) and the contributor digest email (4.2).

4.1.3. Clip Usage Count Display

- `timesUsed` returned from `/api/audio/info` directly from `audio.*` column
- Audio View and Audio Edit display of "Used X times" deferred to the UI/UX plan

4.2. Recipe Usage Tracking

Recipes vary significantly in the mixes they produce — some consistently generate short ambient pieces, others produce long-form compositions that can run an hour or more. Without tracking actual mix durations, the `RecipeSelector` has no way to account for this variation, and recipes that produce longer mixes occupy disproportionate airtime relative to how often they are selected.

4.2.1. Infrastructure Changes

4.2.2. Tracking Service

SQL

```
-- New table: tracks each mix produced by a recipe
CREATE TABLE recipeUsage (
```

```

usageID  INT      NOT NULL AUTO_INCREMENT,
recipeID INT      NOT NULL,
duration FLOAT    NOT NULL,
usedAt   DATETIME DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (usageID),
KEY idx_recipeID (recipeID)
);

-- New columns: cached play count and running average mix duration
ALTER TABLE recipes ADD COLUMN timesUsed INT DEFAULT 0;
ALTER TABLE recipes ADD COLUMN avgDuration FLOAT DEFAULT NULL;

```

The existing `RecordKeeper` service is extended to handle recipe tracking alongside clip tracking. After `mixDuration` is resolved, `RecordKeeper`:

- Increments `recipes.timesUsed` by 1
- Inserts a row into `recipeUsage` recording the `recipeID`, `duration`, and timestamp
- Updates `recipes.avgDuration` as a weighted running average:

None

```

avgDuration = (avgDuration × historyWeight + mixDuration) /
(historyWeight + 1)

```

- If `avgDuration` is `NULL` (first run), `mixDuration` is stored directly rather than treating `NULL` as zero.

The history weight is stored in config (default: 10), meaning after roughly 10 runs the estimate is reasonably stable. Editing a recipe to use shorter or longer clips will gradually shift the average to reflect real-world behavior.

4.2.3. Considerations

- *Cold start*: `avgDuration` is `NULL` for all recipes until their first mix completes. `RecipeSelector` treats `NULL` as neutral — no duration penalty applied until data accumulates.

- *Variability*: `avgDuration` reflects actual mix durations, which vary run-to-run depending on clip selection. This is intentional — it captures real-world behavior rather than a theoretical estimate.
- `recipeUsage` and `avgDuration` feed recipe length weighting in `RecipeSelector` (3.5) and are available for future analytics.

4.2.4. Recipe Usage Display

- `timesUsed` and `avgDuration` available from the recipes API for display in Recipe View, Recipe Edit, and Recipe List (deferred to UI/UX plan).

4.3. Recipe Selection Weighted by Average Duration

Recipes vary considerably in the mixes they produce, and without accounting for that variation the `RecipeSelector` treats all candidate recipes as equally costly in terms of airtime. A recipe that consistently generates 60-minute mixes will occupy twice as much stream time as one that generates 30-minute mixes, even if both are selected at the same rate. Over a 24-hour period this compounds: the stream skews toward the output of a few long-running recipes rather than cycling broadly through the available library.

This task adds a duration-based subscore to the `RecipeSelector` that down-weights recipes whose `avgDuration` is high relative to the current candidate pool, giving shorter-producing recipes a proportionally higher chance of selection.

4.3.1. Infrastructure Changes

At selection time, `RecipeSelector` queries `MIN` and `MAX avgDuration` across the eligible recipe pool. Each candidate recipe receives a duration subscore:

```
durationScore = (maxDuration - avgDuration) / (maxDuration - minDuration)
```

This yields a value in [0, 1] where recipes near the minimum average duration score 1.0 and recipes near the maximum score 0.0. The score is combined with existing subscores (recency, classification diversity) as a weighted sum. The weight scalar is stored in config under `recipes.durationScoreWeight`.

Recipes with `NULL avgDuration` — those that have not yet generated a mix — receive a neutral score (0.5) so they are neither favored nor penalized before data accumulates.

4.3.2. Considerations

- **One factor among many:** A long-average recipe is nudged toward lower selection probability, not disqualified — it will still be chosen when other factors favor it.
- **Cold start:** `NULL avgDuration` scores 0.5 (true neutral). Using 1.0 would unfairly favor unproven recipes over established shorter ones.
- **Degenerate pool:** If all eligible recipes share the same `avgDuration`, score defaults to 0.5 to avoid division by zero.
- **Depends on `avgDuration` data from 3.2;** will self-calibrate as more runs accumulate.

4.4. Clip Selection Weighted by Times Used

The clip pool for any given recipe may contain many qualifying clips, but without usage-awareness the `ClipSelector` draws from it without regard to how often individual clips have already aired. Popular or well-tagged clips can dominate the rotation while lesser-used clips rarely surface.

This task adds a usage-based weight to clip selection so that clips with lower `timesUsed` counts are proportionally more likely to be chosen, broadening the effective rotation without excluding any clip entirely.

4.4.1. Infrastructure Changes

At clip selection time, `ClipSelector` computes MIN and MAX `timesUsed` across the eligible clip pool. Each candidate clip receives a usage subscore:

```
usageScore = (maxUsed - timesUsed) / (maxUsed - minUsed)
```

Clips with `timesUsed = 0` or `NULL` score 1.0 (never played = highest priority). The score is combined with the existing subscores (recency, tag match) as a weighted sum. The weight scalar is stored in config under `audio.usageScoreWeight`.

Edge cases:

- **Never-used clips:** Score 1.0 — new uploads get natural exposure before any data accumulates.
- **Degenerate pool:** If all clips have the same `timesUsed`, score defaults to 0.5 to avoid division by zero.
- **Depends on `timesUsed` data from 3.1;** will self-calibrate as more runs accumulate.

4.4.2. Considerations

- **Depends on `timesUsed` data from 3.1;** will self-calibrate as more runs accumulate.

- **Degenerate pool:** If all eligible clips have the same `timesUsed`, score defaults to neutral to avoid division by zero.
- **Not a hard exclusion:** Heavily-used clips remain eligible — they are just less likely to be chosen when fresher alternatives exist.

4.5. New Audio Filters

We are adding five new filters to extend the expressive range of recipes, and auditing and testing all existing filters to ensure they work as expected and intended. This is also a good opportunity to document all DriftConditions recipe filters and options comprehensively.

`label` is a new optional key on track objects (`label: "vocals"`). It is ignored by all existing logic and requires no recipe migrations — existing recipes continue to work unchanged. New recipes can add `label` where `duck()` needs a named target.

`crossfade` and `fadeout` have the highest leverage since they're already baked into active recipes — implementing them will immediately improve 7 mixes with no recipe changes needed.

4.5.1. Effect Application Order

MixEngine now enforces a canonical effect application order regardless of the order effects appear in a recipe. Effects are sorted into four categories before processing: *structural* (`loop`, `crossfade`, `fadeout`, `trim`, etc.), *level* (`norm`), *color/texture* (`backward`, `faraway`, `telephone`), and *dynamic volume* (`wave`, `duck`). This prevents recipe authors from accidentally writing effects in an order that defeats their intent — for example, placing `norm` after `wave` would re-normalize the audio and undermine the volume modulation. With enforced ordering, `["wave(bridge)", "norm", "loop"]` and `["loop", "norm", "wave(bridge)"]` produce identical results: loop first, norm second, wave last. Recipe authors no longer need to think about ordering — the engine does it correctly.

4.5.2. Filter: repeat(n)

`repeat(n)` reuses the audio clip selected for the `n`th clip in the same track, rather than triggering a new random selection. This allows recipe authors to create call-and-response structures, motif repetition, or symmetrical compositions without relying on chance to pick the same clip twice.

`n` is 0-based and scoped to the current track. `repeat(0)` refers to the first clip in the track, `repeat(1)` the second, and so on. Silence slots count toward the index but cannot be the target of a repeat — if `n` points to a silence or is out of range, the clip falls through to normal random selection with a warning logged.

4.5.2.1. Implementation

Implementation is entirely in `ClipSelector.selectAudioClips()`. As clips are selected within a track, a `selectedClips` array accumulates results in order. When a clip carries a `repeat(n)` effect, `ClipSelector` copies the audio metadata from `selectedClips[n]` directly onto the current clip and skips DB selection. Because the clip object is fully populated before `RecordKeeper` and `MixEngine` run, both stages see it as a normal clip — no changes are required downstream.

4.5.2.2. Playlist behavior

Because `RecordKeeper` deduplicates heard clips by `audioID`, the repeated clip appears in the playlist exactly once regardless of how many times it is used in the track.

4.5.2.3. Limitations

`repeat(n)` is track-scoped; there is no cross-track repeat. The effect is processed at clip selection time, so it cannot repeat a clip from a future slot.

4.5.3. Filter: fadeout(n)

`fadeout` applies a smooth volume ramp-down at the end of a track or clip, so the audio fades to silence rather than cutting off abruptly. This is useful for ambient beds, looped music tracks, and narrative closings where a hard cut would feel sonically jarring. The optional parameter `n` sets the fade duration in seconds; the default is 3.

4.5.3.1. Implementation

Implementation is split between `_buildTrackFilters()` and `_buildClipFilters()` in `MixEngine.js`, with a shared `_fadeoutEffect()` helper that inserts an `afade t=out` filter into the chain.

Clip-level: Applied immediately in `_buildClipFilters()` using the known `clip.duration`. Start time is `clip.duration - fadeDuration`.

Track-level: Deferred. Looped tracks have no natural end — their effective duration is not known until `_determineMixDuration()` runs. When a `fadeout` effect is encountered in `_buildTrackFilters()`, a pending entry `{ inputLabel, baseLabel, fadeDuration, trackDuration }` is pushed to `this.pendingFadeouts[]`. After mix duration is resolved, `_applyPendingFadeouts(mixDuration)` inserts the `afade` filter and patches `trackFinalLabels` so the mix filter receives the faded label. For looped tracks, start time is `mixDuration - fadeDuration`; for finite tracks, `trackDuration - fadeDuration`.

4.5.3.2. Limitations

If `fadeDuration` exceeds the track or clip duration, the filter is skipped with a warning and the audio plays unmodified. Clip-level `fadeout` on a looped clip is not supported — clip duration is `Infinity` at filter-build time — and is skipped with a warning; use track-level `fadeout` instead. `fadeout` should be the last effect in the effects list, as any subsequent effect will operate on already-fading audio.

4.5.4. Filter: duck(n)

`duck(n)` reduces the volume of the current track whenever the specified sidechain track has signal. This is the classic radio/podcast ducking pattern: a music bed quietly retreats when a narrator speaks, then recovers when they stop. The parameter `n` is either a zero-based track index or a track label string (e.g. `duck(0)` or `duck(narrative)`).

The degree of ducking is controlled by four parameters in `config.ffmpeg.filters.duck`: `threshold` (signal level that triggers ducking, default -30dB), `ratio` (compression ratio at trigger, default 20), `attack` (ramp-down time in ms, default 200), and `release` (ramp-up recovery time in ms, default 1000). At `ratio` 20 the ducked track drops to roughly 10% of its original volume while the sidechain is active.

4.5.4.1. Implementation

Implementation is split across `_buildTrackFilters()` and `_applyPendingDucks()` in `MixEngine.js`, with a `_duckEffect()` helper that inserts the `sidechaincompress` filter.

When `duck(n)` is encountered in `_buildTrackFilters()`, a pending entry `{ trackNum, baseLabel, sidechainRef }` is pushed to `this.pendingDucks[]`. `sidechainRef` is either an integer (direct index) or a string (resolved against `this.trackLabels[]` at apply time). Processing is deferred because the sidechain track's final label may not yet be settled when the ducked track is being built.

After all tracks are built, `_applyPendingDucks()` processes each pending entry:

1. Resolves the sidechain track index from `sidechainRef`.
2. Calls `asplit` on the sidechain track's current final label, forking it into a mix copy and a sidechain copy. The mix copy replaces the sidechain track's entry in `trackFinalLabels` so the sidechain audio still appears in the final mix.
3. Inserts a `sidechaincompress` filter taking the ducked track and the sidechain copy as inputs. The output label replaces the ducked track's entry in `trackFinalLabels`.

The `loudnorm` filter applied to foreground clips during track building ensures the sidechain signal reliably crosses the compressor threshold regardless of the source clip's original level.

4.5.4.2. Limitations

`duck(n)` is a track-level effect only; it cannot be applied at the clip level. A track can only be ducked by one sidechain source. The sidechain track must exist in the recipe --- an out-of-range index or unrecognized label is skipped with a warning and the track plays unducked. The sidechain track itself is not affected in level; only the track carrying the `duck()` effect is compressed.

4.5.5. Filter: `wave()`

`wave()` applies a slowly oscillating volume envelope to a track, creating the impression of a signal drifting in and out — a radio-tuning effect, a signal fading through interference, or two tracks alternating presence. While `wave()` existed previously as a collection of hardcoded formula variants, it has been significantly redesigned around a single parameterized base formula with composable modifiers.

The base formula is a harmonic cascade of three cosines at decreasing frequency and amplitude, clamped to [0, 1]. All wave characteristics — frequency family, amplitude, polarity, floor — are parameters. Named presets select frequency families; modifiers patch individual parameters on top.

Presets select the oscillation period:

- `wave()` or `wave(default)` — active, noticeable oscillation
- `wave(slow)` — gentler, longer period
- `wave(slower)` — very long period, barely perceptible movement

Modifiers are composable in any order:

- `counter` / `inverse` — inverts polarity; the wave is the complement of the base preset. Useful for pairing two tracks so one is loud when the other is quiet.
- `soft` — reduces amplitude and raises the floor; the track never fully disappears. Useful for subtle background presence.
- `lifted` — raises the floor so the wave never drops to zero; useful for tracks that should stay audible throughout.
- `bridge` / `transition` — peaks at the crossover points between a `wave()` track and its `wave(counter)` complement. Useful for interference or static that surges during transitions between two alternating tracks.

Modifiers combine freely: `wave(slow, soft)`, `wave(slow, counter)`, `wave(slower, lifted)`, etc.

4.5.5.1. Implementation

The resolver lives in `_waveEffect()` in `MixEngine.js`. It reads `config.exprs3` — a single `base` formula string plus a `presets` object. At call time, it identifies the preset (first param matching a preset name, defaulting to `default`), collects remaining params as modifiers, patches the preset's parameter object accordingly, then substitutes all parameter values into the base formula using word-boundary regex replacement. The resolved formula string is passed directly to ffmpeg's `volume` filter with `eval: frame`.

`bridge` is a special case: it computes both the lead (normal) and counter (inverted) expressions, then combines them as

```
min(1, max(0, 4*(0.5-abs(0.5-{lead}))* (0.5-abs(0.5-{counter}))+0.25)).
```

4.5.5.2. Recipe migration

All production recipes using legacy wave variant names (`subtle`, `inverseNoise`, `subtleInverse`, `noise2`, `fadeInNOut`, etc.) have been updated to the new syntax. The old `exprs` config block is retained for backward compatibility with any recipes not yet migrated; `exprs2` has been removed.

4.5.5.3. Limitations

Wave period is fixed at mix-build time — it does not adapt to clip or mix duration. Very short clips may not complete a full oscillation cycle. `wave()` is a volume envelope only; it does not affect panning or spatial position.

4.6. Automatic Audio Analysis (Essentia.js)

Music clips benefit from BPM and key tags — they make recipes more musically coherent and allow contributors to discover related clips by tag. Manual tagging is impractical at scale. This section adds an automated analysis pipeline that runs nightly and emits tags directly into the clip's existing tag array, indistinguishable from contributor-supplied tags.

4.6.1. Analysis Script

`experiments/essentia/analyze.js` is a standalone Node.js subprocess that accepts a file path and outputs a JSON tag array to `stdout`. It uses the `Essentia.js` WASM library (in-process, no Python, no native compilation) with `audio-decode` for file decoding.

To avoid skewed results from intros and outros, it analyzes up to 180 seconds from the middle of the file, mixed down to mono. Three algorithms run in sequence:

- **BPM** — `RhythmExtractor2013` (multifeature mode). Tags as `{n}-bpm` when confidence ≥ 0.5 . If detected BPM exceeds `120`, also emits a halved tag (e.g. `64-bpm` alongside `128-bpm`) to catch double-time detector latch.
- **Key** — `KeyExtractor`. Tags as `{note}-{major|minor}-key` (e.g. `g-minor-key`, `b-flat-major-key`) when strength ≥ 0.5 . Handles sharps and flats in the note name.
- **Danceability** — `Danceability`. Tags as `danceable` when score ≥ 1.1 (scale 0–~3).

Diagnostics (raw scores, confidence, pass/fail) are written to `stderr` so the runner can log them without contaminating the `stdout JSON`. The script exits non-zero on analysis failure so the runner can distinguish errors from empty-tag results.

4.6.2. Queue and Runner

Upload triggers queuing automatically — no manual action required from contributors. On upload (and on edit), `audioRoutes.js` checks whether the clip's classification includes any entry from `config.audio.musicAnalysisClassifications` (instrumental, vocalmusic, ambient). If so, it writes `needs-audio-analysis` to the clip's `internalTags JSON` column.

`audioAnalysisRunner.js` processes the queue:

1. Queries all non-trashed audio records containing `needs-audio-analysis` in `internalTags`.
2. Spawns `analyze.js` as a child process for each, passing the full file path.
3. On success: merges the returned tags (deduped) into the clip's existing tags array; swaps `needs-audio-analysis` → `audio-analyzed` in `internalTags`.
4. On failure: swaps `needs-audio-analysis` → `analysis-error` instead. Failures are retryable by re-adding `needs-audio-analysis` to the clip's internal tags.

`internalTags` are never shown to contributors — they are a pipeline state mechanism only.

4.6.3. Scheduling

The runner fires nightly via a `systemd` one-shot service (`audio-analysis.service`) and timer (`audio-analysis.timer`), managed the same way as the digest runner. Entry point is `scripts/run-audio-analysis.js`, which must be invoked with `cwd = AdminServer/` so the config symlink resolves.

4.6.4. Considerations

- **Tag format** — Analysis tags use the same format as contributor tags (`102-bpm`, `g-minor-key`) and land in the same tags column. `MixEngine` and the UI see no distinction between auto-generated and hand-written tags.
- **Classification gating** — Non-music clips (spoken, environmental, archival, etc.) are never queued. Analysis on speech or field recordings would produce meaningless BPM and key output.
- **Idempotency** — Re-running the runner on an already-analyzed clip is safe; `audio-analyzed` clips are excluded from the queue query. To re-analyze a clip (e.g. after threshold tuning), swap its internal tag back to `needs-audio-analysis`.
- **Double-time correction** — `Essentia.js`'s rhythm extractor sometimes latches onto a subdivision rather than the true beat. Emitting both the detected BPM and its half-value gives recipe authors and contributors both options without forcing a judgment call in the analysis script.

4.7. Transcoding Large Files

Audio files uploaded to DriftConditions vary widely in format and bitrate — lossless `WAVs`, high-bitrate `MP3s`, and `AIFF` files can be tens or hundreds of megabytes. These large files impose no quality benefit for streaming: mixes are composed offline by the `MixEngine` using full-resolution source files, and the resulting mix is independently normalized before it reaches Liquidsoap. The overhead falls entirely on transmission — serving the admin UI, cover backfill requests, and any future public file access.

This task introduces a lightweight transcoding pipeline that detects oversized files at upload time, schedules them for transcoding to 192kbps MP3, and backfills the existing library. Every audio record ends up with a clear status tag so the population can be searched, audited, and re-processed if the target bitrate changes.

4.7.1. Detection Logic

A file is a transcode candidate if both conditions are true:

- **File size exceeds the threshold** — default 10 MB, stored in config as `audio.xcodeFileSizeThresholdMB`
- **Bitrate exceeds the target** — default 192 kbps, stored in config as `audio.xcodeBitrateThresholdKbps`

Size is the primary gate because that is what drives transmission cost. A large uncompressed file (`WAV`, `AIFF`, `FLAC`) nearly always exceeds both thresholds. A file already at or below 192

kbps is not transcoded regardless of size — this prevents unnecessary re-encoding of files that are already appropriately compressed. Small files, regardless of format, are not transcoded: a short WAV clip may be technically uncompressed but imposes no meaningful transmission burden.

Both thresholds live in config so they can be adjusted without code changes if storage or bandwidth requirements shift.

4.7.2. Tagging Convention

All audio records are tagged in `internalTags` to reflect their transcode status. At upload time, `needsTranscode()` runs immediately and writes one of the first two tags based on the threshold check. The tag is the result of the decision — a file carrying `xcode-needed` has already been evaluated and confirmed to require transcoding.

- `xcode-needed` — evaluated at upload; file exceeds both thresholds and transcode is required
- `xcode-not-needed` — evaluated at upload; file is within thresholds; no action required
- `xcode-completed` — transcode has run and `audio.filename` updated
- `xcode-192` — records the target bitrate used (written alongside `xcode-completed`)

The bitrate tag (`xcode-192`) is written separately so that if the target bitrate changes in the future, all records transcoded at the old rate can be found by searching for `xcode-192` without `xcode-[new bitrate]` and re-queued without a full library scan.

At the end of a complete backfill run, every audio record will carry exactly one of `xcode-not-needed` or `xcode-completed`. Records carrying any `xcode-*` tag are skipped on subsequent runs, making the job safely re-runnable.

4.7.3. Infrastructure Changes

Two utility functions are added to `AdminServer/utils/xcodeUtils.js`:

`needsTranscode(audioRecord, filePath)` — evaluates whether the file at `filePath` exceeds both thresholds. Uses `ffprobe` (bundled with `ffmpeg`) to read container bitrate without decoding the file. Writes `xcode-needed` or `xcode-not-needed` to `audio.internalTags` and returns a boolean.

`transcodeFile(audioRecord, filePath)` — transcodes the file to 192kbps MP3 using `ffmpeg`. The output is written side-by-side with the original using a `-xcode` filename suffix:

- `2024/06/long-recording.wav` ← original retained
- `2024/06/long-recording-xcode.mp3` ← transcoded output

After a successful transcode:

- `audio.filename` is updated to point to the `-xcode` file
- `internalTags` is updated: `xcode-needed` replaced by `xcode-completed` and `xcode-192`
- The original file is retained to allow spot-checking and manual rollback

At upload time, `needsTranscode()` is called immediately after the file is saved. If transcoding is needed, `transcodeFile()` runs before the upload response is returned, so `audio.filename` always reflects the file that will actually be served.

4.7.4. Backfill Job

`AdminServer/jobs/run-xcode-backfill.js` iterates all audio records that do not already carry `xcode-needed`, `xcode-not-needed`, or `xcode-completed` in `internalTags`. For each untagged record it:

1. Resolves the file path from `audio.filename`
2. Calls `needsTranscode()` to evaluate and tag the record
3. If `xcode-needed`, calls `transcodeFile()` to produce the output and update `audio.filename`

The job skips already-tagged records, so it is safe to run multiple times and can be interrupted and resumed without duplication. A companion runner in

`AdminServer/utils/xcodeRunner.js` provides the iteration and logging logic, following the same pattern as `run-cover-backfill.js`.

Edge cases:

- **File not found:** Logged as a warning; record tagged `xcode-error` for manual review. Job continues to the next record.
- **ffprobe failure:** Treated as unknown bitrate; record tagged `xcode-needed` to be conservative.
- **Transcode failure:** Logged as an error; original `filename` and `xcode-needed` tag are retained so the file will be retried on the next run.
- **Already-optimized files:** Files within both thresholds receive `xcode-not-needed` and are never touched again unless manually cleared.

4.7.5. Scheduling

The `xcode runner` follows the same `systemd` timer pattern as the `audio analysis job`. Two phases of scheduling apply:

Initial backfill. Run once manually after deployment to process the existing library:

```
Shell
cd /home/debian/driftconditions/AdminServer && node
jobs/run-xcode-backfill.js --prod
```

The job is re-runnable — it skips any record already carrying an `xcode-*` tag, so it can be interrupted and restarted safely.

Nightly sweep. After the backfill completes, a nightly `xcode.timer / xcode.service` `systemd unit` runs the same job on a daily schedule (suggested: 10:00 UTC, offset from the audio analysis run at 08:00). Its purpose is to pick up files tagged `xcode-needed` at upload time that have not yet been transcoded, and to retry any records tagged `xcode-error` from a prior failed run.

Because detection runs synchronously at upload (tagging the record immediately), the nightly job has no detection work to do for new uploads — it only processes confirmed `xcode-needed` records. Once the library is fully backfilled, nightly runs are expected to be short.

4.7.6. Considerations

- **No effect on mix quality.** The `MixEngine` reads source clips from `audio.filename`. After transcode it picks up the 192kbps file automatically — but mixes are composed offline and the output mix is independently normalized, so source bitrate has no bearing on the final stream.
- **Original files retained.** The `-xcode` output is written alongside the original rather than replacing it in place. This allows spot-checking and manual rollback of `audio.filename` if a problem is found. Bulk original cleanup can be added as a future step once confidence is established.
- **Bitrate detection via `ffprobe`.** Reads container metadata without decoding — fast even for large files. For VBR files, the reported bitrate is the container's declared average, which is the appropriate measure for transmission cost.
- **Config-driven thresholds.** Both `xcodeFileSizeThresholdMB` and `xcodeBitrateThresholdKbps` are read from config at runtime, not hardcoded.

5. Implementation: Backend Infrastructure

These tasks strengthen the core operational foundations of the station, reduce long-term operational risk, and improve the contributor experience through automated communication.

5.1. Audit Table Instrumentation

An `audit` table exists in the database but is currently empty — no code writes to it. This task instruments the auth and user management routes to write records for key security and administrative events.

Events to log: successful and failed sign-ins, sign-outs, user role changes, user status changes, password resets, and `OAuth` account linking. Each record should include the acting user, the affected user (if different), the event type, a timestamp, and any relevant metadata.

This provides a basic operational audit trail for debugging, moderation review, and security incident response.

5.2. Contributor Digest Email System

Rather than individual per-approval emails, a periodic digest re-engages contributors with a summary of their recent activity on the station. A contributor who submits multiple clips in one session receives one digest, not multiple emails.

Digest content (personalized per user):

- Username and link to profile
- Current role
- Member since date
- Since last digest: clips approved, clips used in recipes (requires 3.3)

Implementation:

- `userComms` table accumulates communication-worthy events (audio approved, clip used, role changed) with a `sentAt` field gating against re-sending
- Digest builder groups pending `userComms` by user and renders a personalized email
- Scheduler runs the digest on a configurable interval (weekly default); only sends if there is at least one pending event
- Uses existing `sendMail` from `utils/mailer.js`

- Users can opt out via a `digestEmail` boolean on the `users` table (default `true`)

Dependencies: The "clips used in recipes" digest row requires 3.3 (clip usage tracking). The digest can launch without it and add that section once tracking is live.

5.3. Logging Consolidation

The system currently writes logs to multiple files in `log/`. This creates operational overhead — log rotation, disk management, and debugging require checking multiple files. The goal is to reduce or eliminate file-based logging in favor of a more manageable approach (stdout/stderr captured by the process manager, or a single consolidated log).

This task involves auditing what is currently written to each file in `log/`, determining what is still needed, and migrating or eliminating the rest. No log data that is currently useful for operations or debugging should be lost without a replacement.

Resolution: Already complete. The file transport in `config/logger.js` was commented out in a prior session, leaving console-only output. Since `adminserver` and `mixengine` run as systemd services, all console output is captured by journald automatically. The `logs/` directory still exists with a few old log files from before the file transport was disabled, but no new files are being written. No code changes required; this item is closed.

5.4. Rename Legacy Infrastructure

The project originated under the name `interference` — a working title that predates the public identity of `DriftConditions`. While the station's public-facing name has been established for some time, the legacy name persists in several places across the development and production infrastructure. This task replaces those references systematically.

Locations requiring rename:

- Production database — named `interference` on the server
- Local development database — named `interference` locally
- `config/config.js` — database name hardcoded as `interference`
- Root `package.json` / `package-lock.json` — `name: interference`
- `.env` (root, local) — `BASEDIR` points to `~/dev/interference/`
- `setupfiles/env.server` — `BASEDIR` and inline comment paths reference `~/interference`
- `setupfiles/env.local` — same
- `scripts/rebuild-client.sh` — directory name check hardcoded to `interference`
- `scripts/user-activity-30-days.sh` — `DB_NAME="interference"` and ssh path
- `AdminClient/src/App.js` — user-visible heading: `<h1>interference</h1>`

- `README.md` — project name and prose references
- Local repo directory — `~/dev/interference/`
- Production server repo directory — `~/interference/`

The new name throughout will be `driftconditions`.

5.4.1. Rename Approach

Code and config changes are straightforward and low-risk. The database rename and directory renames require more care and should be done in a deliberate order. The recommended sequence is:

1. Update all code and config references (`config.js`, `package.json`, scripts, env files, `App.js`, `README.md`)
2. Rename the local database (`interference` → `driftconditions`) and verify locally
3. Deploy code changes to the server
4. Rename the production database on the server
5. Rename the production repo directory (`~/interference` → `~/driftconditions`) and update any symlinks or service files that reference it
6. Rename the local repo directory last (or leave it — it has no runtime effect)

5.4.2. Interruption Mitigation

The production database rename and directory rename are the two steps most likely to cause a service interruption. Both should be treated as a brief, planned downtime.

Database rename: `MySQL` does not support a single `RENAME DATABASE` command. The safe approach is to create a new database, dump and restore the data, verify, then drop the old name. The `MixEngine` and `AdminServer` must both be stopped during the restore to prevent writes to the old database. Total downtime should be under five minutes.

Directory rename on the server: All service files, reverse proxy configs, and environment configs that reference `~/interference` as a path are hosted in `setupfiles/` and symlinked into place on the server. This means every path reference can be updated here in the source before deployment. The complete set of files requiring updates is: `env.server`, `adminserver.service`, `mixengine.service`, `liquidsoap.service`, `Caddyfile.server`, and `icecast-server.xml`. Once those are updated and deployed, the service is stopped, the directory renamed, and the service restarted in a single operation.

Rollback: Until the old database is dropped and the old directory is removed, rollback is a matter of reverting the config file changes and restarting services. Drop and cleanup should not happen until everything has been verified running under the new name.

5.5. Audit Recordkeeping

The `audit` table records consequential state changes — role promotions, status changes, permission edits, moderation decisions, password resets — that admins can query when something goes wrong or needs review. It is not an access log; routine reads and list queries are out of scope.

5.5.1. Schema Change

The existing table was missing a payload column, now added:

SQL

```
ALTER TABLE audit
  ADD COLUMN payload TEXT DEFAULT NULL
  AFTER actionType;
```

`TEXT` rather than MySQL's `JSON` type — consistent with how the rest of the codebase stores structured data (e.g. `userComms.payload`, `roles.permissions`). Parsing is cheap; MySQL's `JSON` column restrictions are not worth the tradeoff.

Full schema:

Column	Type	Notes
<code>auditID</code>	int AUTO_INCREMENT	PK
<code>tableName</code>	varchar(255)	e.g. <code>users</code> , <code>roles</code> , <code>audio</code>
<code>recordID</code>	varchar(255)	affected row's PK
<code>actionType</code>	varchar(32)	e.g. <code>role_change</code> , <code>audio_approved</code>

<code>payload</code>	TEXT	JSON string: { <code>before</code> , <code>after</code> , <code>meta</code> }
<code>actionBy</code>	int	userID of the actor (FK → users)
<code>actionDate</code>	datetime DEFAULT NOW()	

5.5.2. System Actor: nobody

A disabled admin account — username `nobody`, `userID = 1` — serves as the actor for system-initiated events that have no human session (signups, password resets via token link). The account has an invalid password hash and `status = Disabled`, so it cannot be used to log in.

5.5.3. Utility Helper: audit.js

`AdminServer/utils/audit.js` provides a thin `logAudit()` helper. Fire-and-forget — audit failures are logged but never propagate to the caller or roll back the primary operation. `actionBy` defaults to `SYSTEM_USER` (`nobody`, `userID 1`) when omitted.

Key design points:

- **Before/after diff only** — record only the fields that changed, not the entire row. Payloads stay small and diffs stay readable.
- **meta for context** — anything that doesn't fit before/after (e.g. `creatorID` on audio events, notification sent flag).
- **No PII in payloads** — email changes are acceptable (before/after is useful for support); passwords must never appear. Callers are responsible for filtering.

5.5.4. Instrumented Routes

Users and roles:

Route	Action types	Notes
-------	--------------	-------

<code>userRoutes</code> <code>/profile/edit</code>	<code>role_change</code> , <code>status_change</code> , <code>profile_edit</code> , <code>password_change</code>	Separate record per change type; before-state fetched before <code>UPDATE</code>
<code>userRoutes</code> <code>/disable</code>	<code>status_change</code>	Before snapshot captured
<code>roleRoutes</code> <code>/update</code>	<code>role_permissions_change</code>	Security fix applied: route was unauthenticated; now enforces <code>verifyToken</code> + <code>roleList</code> permission

Auth events:

Route	Action type	Notes
<code>authRoutes</code> <code>/signup</code>	<code>user_created</code>	<code>actionBy = nobody</code> ; after snapshot captures username, email, name, location
<code>authRoutes</code> <code>/reset-password</code>	<code>password_reset</code>	<code>actionBy = nobody</code> ; token-gated self-service

Audio and recipes (symmetric):

Route	Action types	Notes
<code>audioRoutes</code> <code>/update</code>	<code>audio_approved</code> , <code>audio_disapproved</code> , <code>audio_status_change</code> , <code>audio_updated</code>	Status changes get their own type; other edits grouped under <code>audio_updated</code> ; <code>creatorID</code> in meta

<code>audioRoutes /trash</code>	<code>audio_trashed</code>	Full before snapshot: title, status, classification, tags
<code>recipeRoutes /update</code>	<code>recipe_approved,</code> <code>recipe_disapproved,</code> <code>recipe_status_change,</code> <code>recipe_updated</code>	Same pattern as audio
<code>recipeRoutes /trash</code>	<code>recipe_trashed</code>	Full before snapshot: title, status, classification, tags

Audio and recipe uploads are not audited — the row itself (`creatorID`, `createDate`, `title`, `status`) is the creation record.

5.5.5. Remaining / Phase 2

Event	Route	Action type
Sign-in (local)	<code>authRoutes /signin</code>	<code>signin</code>
Sign-in (OAuth)	<code>authRoutes /callback/*</code>	<code>signin_oauth</code>
Logout	<code>authRoutes /logout</code>	<code>signout</code>
Password reset requested	<code>authRoutes /forgot-password</code>	<code>password_reset_request</code>
OAuth provider linked	<code>authRoutes /callback/*</code> (existing user)	<code>oauth_link</code>

Sign-in/out events are high-volume. Add a retention policy before enabling them — sign-in rows will accumulate quickly at any meaningful user base.

6. Stretch Goals

6.1. Thumbs Up/Down

This feature implements a listener rating mechanism within the playlist component. A listener can give a thumbs up or thumbs down to the currently playing mix. The response is recorded and associated with that playlist entry.

A thumbs-up rating also flags the playlist so it is not pruned by the prune mixes job, preserving well-received mixes in the archive.

Completed in **DriftConditions Spring 2026 Player Upgrade Plan**.

6.2. User List Quick Edit

Status: Complete. An inline quick-edit row was added to the User List page in a prior commit. Role and Status dropdowns are available without leaving the page.

6.3. YouTube / Twitch Stream Output

This stretch goal investigates the technical feasibility of routing the generated stream to YouTube and/or Twitch in addition to the existing Icecast output. The deliverable is a feasibility assessment — what is required, what the constraints are, and whether it is worth pursuing — not necessarily a working implementation.

7. Appendices

7.1. Appendix A — Database Schema Diff

See individual sections.

7.2. Appendix B — Server Changes

See individual sections.

7.3. Appendix C — Future Enhancements and Technical Debt

See stretch goals left uncompleted.

7.4. Appendix D — Document Style Guide

This section details the standardized formatting applied to the document.

7.4.1. Headers and Section Titles

This style dictates the formatting for all section titles and headings (Heading 1, Heading 2, Heading 3, etc.) to ensure clear hierarchy and visual distinction from body text. All headers must be formatted exclusively using these standard Google Docs heading styles rather than custom text styles.

- **Font Family:** Arial
- **Font Size:** Varied (e.g., 20pt for Heading 1, 16pt for Heading 2, 14pt for Heading 3)
- **Font Weight:** Bold
- **Application:** Applied exclusively to section titles and subtitles, using the designated Docs Heading styles.
- **Structural Patterns:** H1 headings (e.g., "1. Goals and Scope") must use the "Heading 1" doc style; H2 headings (e.g., "1.1. Goals") must use the "Heading 2" doc style; and this hierarchical pattern must be followed for all subsequent heading levels.

7.4.2. Body Paragraphs and Standard Prose

This section defines the baseline formatting for all narrative and descriptive text in the document. Standardized prose ensures optimal readability and consistency across all sections, distinguishing it clearly from headings and technical lists.

- **Font Family:** Arial
- **Font Size:** 11pt
- **Line Spacing:** 1.15
- **Spacing (Before Paragraph):** 0pt
- **Spacing (After Paragraph):** 12pt
- **Application:** Applied exclusively to normal body text, excluding headings, titles, and list items.

7.4.3. Inline Technical References

This style is used for all short code references embedded within the main text, such as variable names, file paths, API routes, and terminal commands. It serves to visually isolate code elements from surrounding narrative text.

- **Font Family:** Roboto Mono (Monospaced)
- **Text Color:** Gemini Green (#188038)
- **Background:** None
- **Use Cases:** Inline code snippets (e.g., `clipUsage` table), variable names (e.g., `audioID`), file paths (e.g., `config/config.js`), and console output text.

7.4.4. Code Block Syntax Highlighting Style: One Light (Notion variant)

A light-background syntax theme derived from the One Light family, originally developed for the

Atom editor. Clean, low-contrast, and readable on white or near-white backgrounds.

Background: Light gray (#f7f7f7 or similar off-white) **Default text:** Dark gray (identifiers, punctuation, table/column names)

Token colors:

- **Keywords** (ALTER, CREATE, ADD, NOT, NULL, ON, DELETE): Medium blue (#0070c1 or similar)
- **Types** (INT, VARCHAR, DATETIME): Bright blue (#0099cc or similar)
- **Numeric literals** (255, 32, 191): Orange-red (#e06c00 or similar)
- **Comments** (-- ...): Muted pink/mauve (#c678a0 or similar)
- **Language label** (SQL): Small-caps, light gray, non-highlighted

Typography: Monospace, medium weight, generous line height. Token colors are distinct but not garish — optimized for readability over visual drama.

SQL

```
-- Existing 'users' table update (only adding utility columns)
ALTER TABLE users
  ADD COLUMN displayName VARCHAR(255),
  ADD COLUMN lastLoginAt DATETIME;

-- New 'userIdentities' table to support multiple linked OAuth accounts
CREATE TABLE userIdentities (
  userId INT NOT NULL, -- Foreign key to the users table
  oauthProvider VARCHAR(32) NOT NULL,
  oauthId VARCHAR(191) NOT NULL,
  PRIMARY KEY (oauthProvider, oauthId),
  FOREIGN KEY (userId) REFERENCES users(userID) ON DELETE CASCADE
);

-- Allow null names since OAuth providers may not supply them
ALTER TABLE users
```